BITS / HEX

- k bits → 2^k values; N items → ceil($\log_2$N) bits
- 1 hex digit = 4 bits; n hex digits = 4n bits = n/2 bytes
- A=1010 B=1011 C=1100 D=1101 E=1110 F=1111
- hex addr n digits → 2^(4n) bytes addressable

0x3ACF → 16 bits, 2 bytes, 64KB addressable
0x7F3A = 0111 1111 0011 1010

Dec→hex: divide by 16, read remainders bottom-up
185÷16=11(B) r9 → **0xB9** | 255÷16=15(F) r15(F) → **0xFF**
To verify: 0xB9 = 11×16+9 = 176+9 = 185 ✓

TWO'S COMPLEMENT

- k-bit signed: −2^(k−1) to 2^(k−1)−1 | 8-bit: −128 to 127
- MSB=1: signed = unsigned_val − 2^k
- Negate: flip all bits, add 1
- **Overflow:** two positives → negative result (or vice versa)

95+60=155 > 127 → overflow. 10011011_2 → 155−256 = **−101**
120+15=135 → 10000111_2 → 135−256 = **−121**
11111110_2 unsigned=254 → signed=254−256=**−2**

4-BIT ENCODING REFERENCE

Bin	Un	2C	1C	Hx	Bin	Un	2C	1C	Hx
0000	0	0	0	0	1000	8	-8	-7	8
0001	1	1	1	1	1001	9	-7	-6	9
0010	2	2	2	2	1010	10	-6	-5	A
0011	3	3	3	3	1011	11	-5	-4	B
0100	4	4	4	4	1100	12	-4	-3	C
0101	5	5	5	5	1101	13	-3	-2	D
0110	6	6	6	6	1110	14	-2	-1	E
0111	7	7	7	7	1111	15	-1	-0	F

1's comp: flip all bits (two zeros: 0000=+0, 1111=−0)
2's comp: flip all bits + 1 (one zero, range −8 to 7)

CUSTOM 6-BIT FLOAT

Layout: [s | e_1e_0 | $f_2f_1f_0$] (sign | 2-bit exp | 3-bit frac)

```
value = (-1)^s × 2^(exp-1) × (1 + f₂×0.5 +
f₁×0.25 + f₀×0.125)
```

010110: s=0, exp=10_2=2, scale=2^1=2, frac=1+0.5+0.25=1.75 → **3.5**

FP bias (general): bias = 2^(e−1)−1 where e = # exp bits
FP32: bias=127 | FP16 (5 exp): bias=15 | 4-bit exp: bias=7
BF16: 8 exp bits → bias=127, wider range than FP16

IEEE 754 FP32: DECIMAL → 32-BIT

Layout: [s (1b) | exp (8b, bias=127) | mantissa (23b)]

```
1. Sign: s=0 (+), s=1 (-)
2. Convert |x| to binary (int part + fraction
part)
3. Normalize: shift binary point → 1.xxx × 2^n
4. Stored exp = n + 127  (write as 8-bit
unsigned)
5. Mantissa = first 23 bits of xxx (zero-pad
right)
```

12.375: s=0 | 12=1100_2, 0.375=0.011_2 → 1100.011_2
→ 1.100011 × 2^3 | exp=3+127=130=10000010_2
→ 0 10000010 10001100000000000000000

Decode: 0 10000010 10001100... → exp=130−127=3
→ 1.100011 × 2^3 = 1100.011_2 = **12.375**

Frac→binary: 0.375×2=0.75(0), 0.75×2=1.5(1), 0.5×2=1.0(1) → .011

INSTRUCTION ALIGNMENT

- b-byte instructions: addresses are multiples of b
- # instructions = mem_size / b
- fraction valid byte addresses = 1/b

8-byte instr, PC=0x4000: next → 0x4008, 0x4010, 0x4018...
2KB memory: 2048/8 = **256 instructions** | valid: 1/8
4-byte ARM, 1KB: 1024/4=**256 instr** | valid: 1/4

POWERS OF 2 — QUICK LOOKUP

```
2^0=1        2^1=2        2^2=4        2^3=8
2^4=16       2^5=32       2^6=64       2^7=128
2^8=256      2^9=512      2^10=1,024   2^11=2,048
2^12=4,096   2^13=8,192   2^14=16,384 2^15=32,768
2^16=65,536 2^17=131,072 2^18=262,144
2^19=524,288
2^20=1,048,576 (≈1M)   2^24≈16M    2^30≈1B
(1GB)

Key: N bits → 2^N values → need
ceil(log₂(N_items)) bits
  70,000 items: 2^16=65536<70000<2^17=131072 →
17 bits
  4KB page = 2^12 → offset=12 bits
  1MB = 2^20 | 1GB = 2^30 | 4GB = 2^32
```

CPI / CPU PERFORMANCE

```
CPI_base = Σ(frac_i × cycles_i)
CPI_stall = CPI_base + mem_frac × stall_per_op
total_cycles = IC × CPI
time = total_cycles / clock_rate
speedup = old_time / new_time
```

4GHz, 1.5B instr: 50%ALU(1cy), 20%ctrl(3cy), 30%mem(2cy+40cy stall)
CPI_base = 0.5+0.6+0.6 = **1.7**
CPI_stall = 1.7+0.3×40 = **13.7** → 1.5B×13.7/4B = **5.14s**
stall→10: new CPI=1.7+3=4.7 → speedup=13.7/4.7=**2.91×**

1B instr, 25% mem, 40cy stall: base=1B, stall=0.25B×40=10B, total=11B
stall%=10/11=**90.9%** | stall→10: new=3.5B → speedup=11/3.5=**3.14×**

3GHz, 2B instr: 40%ALU(1), 30%mem(5), 30%ctrl(2): (no stall)
CPI=0.4×1+0.3×5+0.3×2=**2.5** | cycles=5B | time=5B/3GHz=**1.67s**
new: 20%ALU/50%mem/30%ctrl → CPI=0.2+2.5+0.6=**3.3** → time=**2.2s**

PIPELINING

- k stages, t = stage time. First: k×t. Each after: +t. Total n: (k+n−1)×t
- Purpose: ↑ throughput — NOT ↓ latency of one instruction
- Branch misprediction at end of stage j: flush stages 1..j−1 (wasted work)

5-stage, 10ms/stage, 10 instr: (5+9)×10 = **140ms**
Misprediction at end of stage 3 (instr 5): flushes 2 stages → +20ms penalty

AMAT — MEMORY HIERARCHY

2-Level (L1 + DRAM):

```
AMAT = L1_time + (1-L1_hit_rate) × miss_penalty
```

3-Level (L1 + L2 + DRAM):

```
AMAT = L1 + L1_miss × (L2 + L2_miss × DRAM)
P(reach DRAM) = L1_miss × L2_miss
NOTE: L2 hit rate is CONDITIONAL on L1 miss
```

L1=2ns(0.8hr), L2=12ns(0.75hr|L1miss), DRAM=100ns:
AMAT = 2 + 0.2×(12 + 0.25×100) = 2+0.2×37 = **9.4ns**
P(DRAM) = 0.2×0.25 = **5%**

L1=4ns(0.8), L2=20ns(0.75), DRAM=200ns:
AMAT = 4+0.2×(20+0.25×200) = 4+0.2×70 = **18ns**
L1→0.9: AMAT=4+0.1×70 = **11ns**

As avg cycles:

```
avg_cycles = hit_time + miss_rate ×
miss_penalty
total_time = num_accesses × avg_cycles /
clock_rate
```

2B accesses, hit=4cy, miss_pen=80cy, hr=0.85, 2GHz:
avg=4+0.15×80=**16cy** → time=2B×16/2B=**16s** | hr→0.95: **8s**, speedup=**2×**

UNIT CONVERSIONS — QUICK REF

```
Bandwidth:  1 Gbps  = 125 MB/s  (÷8 bits/byte)
            1.6 Gbps = 200 MB/s
            800 Mbps = 100 MB/s

Storage:    1 KB = 1,024 B    1 MB = 1,048,576
B
            1 GB = 10^9 B (problems use this)
OR  2^30 B
            → use 10^9 unless problem says 2^30

Time:       1 ms = 10^-3 s    1 µs = 10^-6 s
1 ns = 10^-9 s
            1 GHz clock = 1 cycle per ns

RPM→speed:  7200 RPM → period=60000/7200=8.33ms
→ avg_rot=4.17ms
            5400 RPM → period=11.11ms →
avg_rot=5.56ms

Bits:       1 byte=8 bits | addr space: n-bit
addr → 2^n bytes
```

CACHE LINES & LOCALITY

```
elem/line = cache_line_bytes / element_bytes
miss_rate (sequential) ≈ 1 / elem_per_line
```

64B line, float64(8B): 8 elem/line → miss≈1/8
64B line, float32(4B): 16 elem/line → miss≈1/16

Row-major (C/NumPy default):

- outer=rows, inner=cols → sequential → **fast** (spatial locality)
- outer=cols, inner=rows → stride=row_length → **cache thrashing**

DATASET SIZE & I/O

```
row_bytes = Σ col_sizes  [int64=8B, float64=8B,
int32=4B]
file_size = row_count × row_bytes
passes = epochs × num_learning_rates
total_IO = file_size × passes
```

500M rows, (UserID int64, Timestamp int64, Value float64): row=24B | file=500M×24=**12GB**
10 epochs × 3 LR = 30 passes → IO = 12×30=**360GB**

FEATURE EXPANSION (PAIRWISE INTERACTIONS)

```
new_cols = C(d,2) = d×(d-1)/2     [d = num
features, no squared terms]
expanded_cols = d + 1_label + C(d,2)

Step A: train on orig   → read orig × epochs ×
LR
Step B: expand features → read orig + write
expanded
Step C: train on expanded → read expanded ×
epochs × LR
Total IO = A + B + C
```

200M rows, d=4 features + 1 label, float64, 3LR × 4epochs:
C(4,2)=6 | orig: 5×8×200M=**8GB** | expanded: 11×8×200M=**17.6GB**
A=8×4×3=96GB | B=8+17.6=25.6GB | C=17.6×4×3=211.2GB
Total = **332.8 GB**

ROW-STORE USEFUL THROUGHPUT

```
useful_data = file_size × (needed_cols /
total_cols)
useful_throughput = useful_data / runtime
bandwidth_util = useful_throughput /
device_bandwidth
```

12GB, 120 equal cols, 1 col needed, 30s, SSD=600MB/s:
useful=12GB/120=100MB | throughput=100/30=**3.33MB/s** | util=3.33/600=**0.56%**

SCHEDULING METRICS

```
Turnaround = completion - arrival
Response   = first_cpu_time - arrival   ← NOT completion
Waiting    = turnaround - burst_time
```

Response ≠ turnaround. Response = when job FIRST gets CPU.

All 3 metrics worked: P1(burst=5,arr=0), P2(burst=3,arr=1), FCFS order
P1: 0→5. P2: 5→8.
P1: TAT=5-0=**5** | response=0-0=**0** | wait=5-5=**0**
P2: TAT=8-1=**7** | response=5-1=**4** | wait=7-3=**4**
Avg TAT=6 | Avg response=2 | Avg wait=2

ALGORITHM COMPARISON

- **FCFS:** simple, no starvation; worst avg TAT if long job first (convoy)
- **SJF:** optimal avg TAT (non-preemptive); starvation risk for long jobs
- **SRTF:** optimal avg TAT (preemptive); best for TAT, high overhead
- **RR:** best avg response time; fair; bad TAT if q too large or small

Response time ranking (best→worst): RR ≥ SRTF > SJF > FCFS
Turnaround ranking (best→worst): SRTF = SJF > RR > FCFS

SCHEDULING ALGORITHM COMPARISON

	FCFS	Round Robin	SJF	SRTF
Preemptive?	❌	✅	❌	✅
Decision point	When CPU free	Every quantum	When CPU free	Every arrival / completion
Track	Arrival	Arrival, remaining, quantum	Arrival, burst	Arrival, remaining
Pick next	Earliest arrival	Next in queue	Shortest burst (arrived)	Shortest remaining (arrived)
Switch when	Current finishes	Quantum expires or finishes	Current finishes	New arrival has less remaining
Tie-break	Earlier arrival	Queue order	Earlier arrival	Earlier arrival

CONTEXT SWITCH OVERHEAD

Each switch adds switch_cost before the next job starts. First job has no overhead.

FCFS P1(20),P2(5),P3(15),P4(10), all t=0, 1ms switch:
P1:0 | P2:21 | P3:27 | P4:43 → wait avg=91/4=**22.75ms**
SJF order P2,P4,P3,P1 → starts 0,6,17,33 → avg=**14ms**

FORK() OUTPUT

Child is a copy of parent at the fork point. Both continue from after fork().

W(A); fork(); W(B); if(child) wait(); W(C);
A: once (before fork) | B: twice (both parent+child)
C: twice (child first if parent waits, then parent)
→ A B B C C

SRTF SIMULATION — REVIEW

```
P1(burst=7,arr=0)  P2(burst=4,arr=2)
P3(burst=1,arr=4)

FCFS:  [P1:0-7][P2:7-11][P3:11-12]
       TAT: 7,9,8  avg=8

SJF:   [P1:0-7][P3:7-8][P2:8-12]  (P3 shorter at t=7)
       TAT: 7,4,10  avg=7

SRTF:  [P1:0-2][P2:2-4][P3:4-5][P2:5-7][P1:7-12]
  t=2: P2.burst(4) < P1.rem(5) → preempt
  t=4: P3.burst(1) < P2.rem(2) → preempt
  t=5: P2.rem(2) < P1.rem(5)  → P2 first
       TAT: P1=12,P2=5,P3=1  avg=6
```

ROUND ROBIN TRACE — REVIEW

```
P1=9, P2=5, P3=3, P4=7  all arr=0, quantum=3
[P1:0-3][P2:3-6][P3:6-9][P4:9-12]
[P1:12-15][P2:15-17][P4:17-20]
[P1:20-23][P4:23-24]
Done: P3=9, P2=17, P1=23, P4=24
Avg TAT = (9+17+23+24)/4 = 18.25
Avg response = (0+3+6+9)/4 = 4.5
  ← response = time of FIRST cpu slice, not completion
```

DISK I/O — HDD MECHANICS

Three components of every I/O:

- **Seek:** arm moves to correct track (1–20ms, avg ~4–8ms)
- **Rotational latency:** wait for sector to spin under head (avg = half rotation)
- **Transfer:** data moves off platter (~100s MB/s, small per-op)

```
rot_period  = 60,000ms / RPM
avg_rot     = rot_period / 2
T_IO        = seek + avg_rot + transfer
transfer    = data_size / bandwidth
bandwidth   = IOPS × IO_size
```

Sequential vs random: sequential can be 100–300× faster than random because contiguous blocks skip seeks and rotational waits.

DISK I/O — CONTIGUOUS VS CHUNKED

```
Contiguous file:
  T = 1×seek + 1×avg_rot + file_size/BW

Chunked access (n chunks of size s):
  chunk_transfer = s / BW
  T_per_chunk    = seek + avg_rot + chunk_transfer
  T_total        = n × T_per_chunk
```

Slide example — 10GB, 7200RPM, seek=8ms, BW=150MB/s, 512KB chunks:
rot=60000/7200=8.33ms → avg_rot=**4.17ms**
chunk_transfer=0.5/150=0.00333s=**3.33ms**
T_per_chunk=8+4.17+3.33=**15.5ms**
n_chunks=10240MB/0.5MB=**20,480**
Total=20480×15.5ms=317s≈**5.3min**
Sequential only: 10240/150=**68s** → chunked is **4.7× slower**

Contiguous vs fragmented — 2GB, 7200RPM, seek=8ms, BW=150MB/s, 512 frags:
avg_rot=4.17ms | transfer=2048/150=13.65s
Contiguous: 12.17ms+13.65s≈**13.66s**
Fragmented: 512×12.17ms+13.65s≈**19.88s** (**1.46× slower**)

STORAGE PERFORMANCE TERMS

- **IOPS:** I/O operations per second the device can sustain
- **Bandwidth:** bytes/sec = IOPS × IO_size
- **Latency:** total time to deliver data (seek+rot+transfer+OS delays)

40GB file, disk BW=800MB/s: $40\times10^9/8\times10^8$=**50s**
Same over 1.6Gbps network (=200MB/s): $40\times10^9/2\times10^8$=**200s**

SSD vs HDD: SSD has no seek/rotation → 100× faster random I/O, 10–100× higher BW. Still block-addressable.

DISK SCHEDULING

SSTF (Shortest Seek Time First): pick pending request closest to current head position. Fast but can starve far-away tracks.
SCAN/C-SCAN (elevator): sweep arm in one direction serving requests, then reverse (or jump back). Prevents starvation; ignores rotational latency.
SPTF (Shortest Positioning Time First): account for BOTH seek AND rotation — pick the request with least total positioning time. Best in practice; done inside drive firmware today.

MULTI-CORE SCHEDULING

SQMS: one shared queue, any core picks next. Simple; lock contention at scale.
MQMS: one queue per core. Less contention; risk of load imbalance.

```
Makespan = time until last job finishes = max(sum of jobs per core)
```

ROW-STORE VS COLUMN-STORE

Row-store: all columns of a row stored together. Good for full-row reads (OLTP). Bad for column-only queries — must scan all columns.
Column-store (Parquet, ORC): each column stored together. Great for column-subset queries — only read needed columns. Better compression (same-type data together).
Parquet vs CSV: compressed (up to 10× smaller), column pruning, schema+metadata embedded, immutable. Cannot edit in-place.

1TB CSV → 130GB Parquet; query 236s→6.78s (34×); scans 99% less data.

PROCESS STATES (QUICK REF)

NEW → READY → RUNNING → BLOCKED → READY

- RUNNING→BLOCKED: process issues I/O syscall
- **BLOCKED→READY: I/O completes**
- RUNNING→READY: timer interrupt (preemption)
- Threads share address space; processes don't
- GIL: CPython = 1 thread at a time (CPU-bound bottleneck)

OS MECHANICS — HOW IT ACTUALLY WORKS

User mode vs Kernel mode: Hardware enforces two privilege levels. Normal programs run in user mode (can't touch hardware directly). OS runs in kernel mode (full access).
Syscall (system call): only way for user program to request OS services. Program executes a **trap** instruction → CPU switches to kernel mode → OS handles the request → returns to user mode. Examples: read(), write(), fork(), malloc() (which calls brk/mmap).
Timer interrupt: hardware timer fires ~every 10ms. CPU automatically jumps to OS interrupt handler (in kernel mode). OS can then: (1) preempt current process (RUNNING→READY), (2) pick another process, (3) return. This is how the OS regains control without process cooperation.
Context switch — step by step:

1. Timer fires → CPU saves PC + registers onto kernel stack
2. OS saves them into the outgoing process's **PCB**
3. Scheduler picks next process from ready queue
4. OS loads new process's PCB (registers, PC) → switch page table (new VA space)
5. Return from interrupt → new process resumes in user mode

PCB (Process Control Block): per-process OS struct storing registers, PC, state, open files, memory map, PID. Lives in kernel memory.

PAGES VS FRAMES — MENTAL MODEL

The core idea: virtual address space is a fiction the OS creates for each process. Physical RAM is real hardware. The OS maps between them using fixed-size chunks.

Page = chunk of **virtual** address space (what the process sees). Pages are the **content** — the data the process wants to store. Numbered 0,1,2,… → that number is the **VPN**. A page doesn't "live" anywhere by itself; it needs a frame to park in.

Frame = slot in **physical** RAM (real hardware). Frames are the **locations** — parking spots that can hold one page's worth of bytes. Numbered 0,1,2,… → that number is the **PFN**.

Relationship: page = content, frame = location. The page table maps one to the other: "page 3 is currently parked in frame 7." A process's pages don't need to be in adjacent frames — page 0 could be in frame 12, page 1 in frame 3. The process never knows; hardware translates invisibly. If RAM fills up, the OS evicts a page from its frame (writes to disk, marks PTE valid=0, frees the frame for someone else). Next access → page fault → OS reloads it into any free frame.

Offset = byte position WITHIN the page/frame. Same in VA and PA — it never changes during translation, because page size = frame size, so the position inside the chunk is identical on both sides.

Page Table Entry (PTE): the mapping record for one page. Stores: PFN (which frame holds this page), **valid bit** (1=in RAM, 0=not in RAM → fault), dirty bit (written since loaded?), permission bits (read/write/exec).

Page Table: array indexed by VPN. One table per process (kernel owns it). Entry at index VPN = PTE for that page.

Translation in plain English:

1. Process generates VA. CPU splits it: upper bits = VPN, lower bits = offset
2. VPN → index into page table → read PTE
3. PTE gives PFN (which frame the page is in)
4. PA = PFN concatenated with offset (same offset, different frame base)

VA=0x2003, page=4KB (0x1000), page table[2]=PFN 5:
VPN=0x2003÷0x1000=**2**, offset=0x2003 mod 0x1000=**3**
PA = 5×4096+3 = 0x5003. Offset (3) unchanged.

VIRTUAL MEMORY BASICS

```
offset_bits = log₂(page_size)
VPN_bits    = address_bits − offset_bits
# pages     = 2^(VPN_bits)  =  VA_space /
page_size

VPN    = virtual_addr ÷ page_size  (upper bits)
offset = virtual_addr mod page_size (lower
bits)

VPN → PFN:  PTE = page_table[VPN]   ← index the
array
            if PTE.valid == 0 → page fault (not
in RAM)
            PFN = PTE.pfn_field      ← read PFN
from PTE

physical_addr = PFN × page_size + offset
```

8-bit VA, page_size=16B (2^4):
offset_bits=4, VPN_bits=4, pages=2^4=16
VA=35: VPN=35÷16=**2**, offset=35 mod 16=**3**
If PFN for VPN 2 = 7 → PA = 7×16+3 = **115**
VA=50: VPN=3, offset=2, PFN=1 → PA=1×16+2 = **18**
32-bit VA, 4KB page: offset=12 bits, VPN=20 bits
entries per process = 2^20 = **1,048,576**

PAGE TABLE SIZE

```
Step 1: offset_bits = log₂(page_size)
Step 2: VPN_bits = address_bits − offset_bits
Step 3: entries per process = 2^(VPN_bits)
Step 4: one_PT_bytes = entries × PTE_size
Step 5: all_PT_bytes = one_PT_bytes ×
num_processes
Step 6: frames_needed = ceil(all_PT_bytes /
page_size)
```

Why frames? Page tables themselves live in physical memory. Each frame holds page_size bytes of page table data.

Worked (the "4 frames" example):
20-bit VA, page=4KB, PTE=4B, 16 processes
Step 1: offset = $\log_2(4096)$ = **12 bits**
Step 2: VPN = 20 − 12 = **8 bits**
Step 3: entries/process = 2^8 = **256**
Step 4: 1 PT = 256 × 4B = **1024 bytes**
Step 5: all PTs = 1024 × 16 = **16,384 bytes**
Step 6: frames = 16384 / 4096 = **4 frames**
24-bit VA, 4KB page, PTE=4B, 8 processes:
VPN=12 bits, entries=4096, 1PT=16KB, all=128KB → frames=128KB/4KB=**32**
VA_space=2^16B, page=2^8B:
offset=8, VPN=8, entries=256/process

PAGE REPLACEMENT POLICIES

FIFO: evict page loaded EARLIEST (track load order)
LRU: evict LEAST RECENTLY USED — update on every HIT too, not just faults
OPT: evict page whose NEXT USE is farthest away (theoretical best)

Simulation steps:

1. Go through reference string left→right
2. HIT: page in frame → no fault; update LRU order
3. FAULT: page not in frame → if empty slot, fill it; else evict per policy
4. Count total faults and hits

PAGE REPLACEMENT EXAMPLE

Reference string: 1 2 3 4 1 2 5 1 2 3 4 5 — 3 frames

FIFO (evict oldest loaded):

```
Ref: 1  2  3  4  1  2  5  1  2  3  4  5
     F  F  F  F  F  F  F  -  -  F  F  -
Frames after each (queue order):
1→[1]  2→[1,2]  3→[1,2,3]  4→evict1[2,3,4]
1→evict2[3,4,1]  2→evict3[4,1,2]
5→evict4[1,2,5]
1=hit  2=hit  3→evict1[2,5,3]  4→evict2[5,3,4]
5=hit
Faults = 9
```

LRU (evict least recently used):

```
Ref: 1  2  3  4  1  2  5  1  2  3  4  5
     F  F  F  F  F  F  F  H  H  F  F  F
After 4: evict LRU=1 → [2,3,4]
After 1: 1∉{2,3,4} → evict LRU=2 → [3,4,1]
After 2: 2∉{3,4,1} → evict LRU=3 → [4,1,2]
After 5: evict LRU=4 → [1,2,5]
1,2: hits | 3: evict LRU=5 → [1,2,3] | 4: evict
LRU=1 → [2,3,4] | 5: F
Faults = 10
```

OPT (evict page used farthest in future):

```
Ref: 1  2  3  4  1  2  5  1  2  3  4  5
     F  F  F  F  H  H  F  H  H  F  F  H
At 4: [1,2,3]. Next: 1@+1, 2@+2, 3@+6 → evict 3
→ [1,2,4]
At 5,6: HIT (1,2 in frames)
At 7: [1,2,4]. Next: 1@+1, 2@+2, 4@+4 → evict 4
→ [1,2,5]
At 8,9: HIT (1,2)
At 10: [1,2,5]. 1,2→never again, 5@+2 → evict 1
→ [2,5,3]
At 11: [2,5,3]. 2,3→never, 5@+1 → evict 2 →
[5,3,4]
At 12: ref=5 → HIT
Faults = 7
```

Key: Thrashing = more page faults than useful work → CPU util drops. Fix: reduce multiprogramming, add RAM.

Larger pages: ↑ internal fragmentation (waste within page). Paging = no external fragmentation.

ABCADBEABC EXAMPLE (3 FRAMES)

Reference string: A B C A D B E A B C

FIFO:

```
A→[A]F  B→[A,B]F  C→[A,B,C]F  A=hit
D→evict A[B,C,D]F  B=hit  E→evict B[C,D,E]F
A→evict C[D,E,A]F  B→evict D[E,A,B]F  C→evict
E[A,B,C]F
Faults = 8
```

LRU:

```
A→[A]F  B→[A,B]F  C→[A,B,C]F  A=hit→order:
[B,C,A]
D→evict B(LRU)[C,A,D]F  B→evict C[A,D,B]F
E→evict A[D,B,E]F
A→evict D[B,E,A]F  B=hit  C→evict E[A,B,C]F
Faults = 8  (tie with FIFO here)
```

OPT:

```
A→[A]F  B→[A,B]F  C→[A,B,C]F  A=hit
D: next use: B@+1,A@+3,C@+5 → evict C →
[A,B,D]F
B=hit  E: D→never → evict D → [A,B,E]F
A=hit  B=hit  C: all→never → evict any →
[A,B,C]F
Faults = 6  (OPT beats FIFO=LRU=8 on this
string)
```

VPN/PFN TRANSLATION — CLASS FORMAT

How the exam gives it: you get a page table as a lookup table (VPN→PFN), a virtual address, and a page size.

Steps:

```
Step 1: VPN    = VA ÷ page_size  (integer
division)
Step 2: offset = VA mod page_size
Step 3: PFN    = page_table[VPN]  ← look up in
the given table
Step 4: PA     = PFN × page_size + offset
```

Review B, Problem 3 — 8-bit VA, page=16B, table:
VPN: 0→5, 1→2, 2→7, 3→1
VA=35: VPN=35÷16=**2**, offset=3 → PFN=table[2]=**7** → PA=7×16+3=**115**
VA=50: VPN=50÷16=**3**, offset=2 → PFN=table[3]=**1** → PA=1×16+2=**18**
16-bit VA, page=256B: VPN=upper byte, offset=lower byte
VA=0x12AB: VPN=0x12=18, offset=0xAB=171
If table[18]=PFN 5: PA=5×256+171=**1451**

PTE size: if PA space=2^20B, 4KB pages → 2^8 frames → 8-bit PFN → round up to 4B per PTE

PAGE OFFSET / VPN BITS PRACTICE

```
Given: VA_space size S, page size P
  → offset = log₂(P)
  → VPN = log₂(S/P) = log₂(S) − log₂(P)

Given: 32KB VA, 512B pages:
  → offset = log₂(512) = 9 bits
  → VPN = log₂(32768/512) = log₂(64) = 6 bits
  → 64 virtual pages
```

VM TRANSLATION FLOW (CONCEPTUAL)

Why VM exists: Each process gets illusion of its own private address space (isolation). OS+hardware prevent one process from touching another's memory. Allows using disk as overflow when RAM is full.

Every memory access — what happens:

1. CPU generates virtual address (VA = VPN + offset)
2. MMU checks **TLB** (hardware cache of recent VPN→PFN translations, ~1 cycle)
3. **TLB HIT:** get PFN → PA = PFN × page_size + offset. Done.
4. **TLB MISS:** MMU walks page table in memory → load PFN into TLB → retry
5. **Page FAULT:** PTE says "not in RAM" → trap to OS → OS finds page on disk → loads into free frame → updates PTE → TLB loaded → resume

TLB: tiny fast associative cache on the CPU die (~32–1024 entries). Stores recently used VPN→PFN pairs. Flushed on context switch (different process = different VA space).

Page fault cost: disk access = millions of cycles. So thrashing (constant faults) kills performance.

Valid bit in PTE: 0 = page not in RAM (trigger fault). 1 = page present, PFN valid. **Dirty bit:** 1 = page was written → must write back to disk before evicting.

AMDAHL'S LAW

```
speedup(n) = 1 / ((1-p) + p/n)
  p = parallelizable fraction
  n = number of workers/cores

max_speedup (n→∞) = 1 / (1-p)

speedup = T_serial / T_parallel
efficiency = speedup / n_workers
```

p=0.75 (75% parallel), n=8 cores:
max = 1/(1−0.75) = 1/0.25 = **4×**
8-core = 1/(0.25 + 0.75/8) = 1/0.34375 = **2.91×**
5-core = 1/(0.25 + 0.75/5) = 1/0.4 = **2.5×** | single-core 80min → **32min**

p=0.9, n=10: speedup=1/(0.1+0.09)=1/0.19=**5.26×**
max=1/0.1=**10×** | efficiency=5.26/10=**52.6%**

T_serial=24min, T_parallel(6 workers)=5min:
speedup=24/5=**4.8×** | efficiency=4.8/6=**80%**

Serial part=10s, parallel=90s, n=9: new_parallel=10s
T_new=10+10=20s | speedup=100/20=**5×**

TASK GRAPH LOWER BOUNDS

```
work       = sum of all task durations
crit_path  = longest path from start to end
(serial deps)
lower_bound = max(work / n_workers, crit_path)
max_speedup = work / lower_bound
idle_time   = n_workers × actual_makespan -
work
```

work=100, crit_path=40, 4 workers:
LB = max(100/4, 40) = max(25, 40) = **40**
max_speedup = 100/40 = **2.5×**
actual makespan=50: idle=4×50−100=**100 worker-units**

work=60, crit=15, 3 workers:
LB=max(60/3,15)=max(20,15)=**20**
max_speedup=60/20=**3×** (linear! possible here)

Spring 26-2: work=70, crit=35 (T1→T4→T6), T3=10 alone
3W: LB=max(70/3,35)=35 → speedup=**2×**, idle=3×35-70=**35**
2W: LB=max(70/2,35)=35 but actual makespan=**40**
(T6 blocked until BOTH T4 and T5 finish → forces extra 5 units)

Critical path = serial bottleneck. Speedup ≤ work/crit even with ∞ workers.

CPU UTILIZATION / EFFICIENCY

```
CPU_util = compute_time / (compute + wait_time)
efficiency = useful_work / total_work_done
```

18s compute, 27s communication overhead:
util = 18/(18+27) = 18/45 = **40%**

Job: 5s useful, 15s total (incl. sync + idle):
efficiency = 5/15 = **33%**

NUMA & FALSE SHARING

NUMA (Non-Uniform Memory Access):

- Multi-socket machines: each socket has "local" RAM
- Accessing remote socket's RAM = k× slower than local

Local=100ns, remote=3× slower:
Thread on Socket 0 reading Socket 1's memory → **300ns**

False Sharing:

- Two threads write **different variables** on the **same cache line**
- Every write forces other core to invalidate its copy → constant line bouncing
- Fix: pad struct so each thread's variable is on its own cache line

Thread A writes x, Thread B writes y. x and y are adjacent ints (4B each) on same 64B cache line → false sharing → repeated cache invalidations even though they write to different variables.

True sharing: threads actually share the same variable (needed for coordination).

PARALLELISM CONCEPTS

SIMD: single instruction, multiple data — one instruction on a vector of values (e.g., add 8 floats at once). Used in NumPy, GPU.

GPU vs CPU: GPU = thousands of simple cores for data-parallel work; CPU = few powerful cores for complex branchy code.

Why NumPy beats Python loops: contiguous typed memory, compiled C kernels, SIMD vectorization, no per-element Python overhead.

Parallel overhead types: (1) **Communication:** cores share results (network/memory bus). (2) **Synchronization:** waiting at barriers/locks. (3) **Load imbalance:** fast workers idle waiting for slow ones. All reduce efficiency below 1.0.

LOCKS & SYNCHRONIZATION

Race condition: two threads read/modify same shared data concurrently. Outcome depends on timing. Classic: both read x=0, both do x++, both write x=1 → lost update (should be 2).

Critical section: code region that must run atomically — only one thread at a time. Solution: protect with a lock.

Mutex (lock): binary flag. thread calls **lock()** to enter — if already held, the caller **sleeps** (blocks). Holder calls **unlock()** when done → one waiter wakes up.

Spinlock: thread busy-loops checking the lock (burns CPU). Use when wait is very short (saves context-switch overhead). Bad if wait is long.

Deadlock: Thread A holds Lock 1, needs Lock 2. Thread B holds Lock 2, needs Lock 1. Both block forever.

Fix deadlock: always acquire locks in the **same global order** (e.g., always lock 1 before lock 2).

Atomic operations: hardware guarantee — read-modify-write completes without interruption (compare-and-swap, fetch-and-add). No lock needed for simple counters.

Condition variable: lets a thread sleep until a condition is true (e.g., buffer non-empty). Used with a mutex: unlock mutex → sleep → re-lock on wakeup.

TASK GRAPH EXAMPLE

Tasks with dependencies — draw the DAG:

```
A=3, B=2, C=4, D=1, E=5
Deps: B after A, C after A, D after B+C, E
after D

Paths:
A→B→D→E = 3+2+1+5 = 11
A→C→D→E = 3+4+1+5 = 13  ← critical path

work = 3+2+4+1+5 = 15
crit_path = 13
2 workers: LB = max(15/2, 13) = max(7.5, 13) =
13
max_speedup = 15/13 ≈ 1.15×

Schedule on 2 workers:
W1: A(0-3) → C(3-7) → D(7-8) → E(8-13)
W2: idle(0-3) → B(3-5) → idle(5-8)
Makespan = 13, idle = 2×13-15 = 11 worker-units
```

AMDAHL: SOLVING FOR P OR N

```
Given speedup S and n workers: solve for p
  S = 1/((1-p) + p/n)
  1/S = (1-p) + p/n = 1 - p + p/n = 1 - p(1 -
1/n)
  p = (1 - 1/S) / (1 - 1/n)

Given speedup S and p: solve for n
  1/S = 1 - p + p/n
  p/n = 1/S - 1 + p
  n = p / (1/S - 1 + p)
```

Observed speedup=3× with 6 cores: p=(1−1/3)/(1−1/6)=(2/3)/(5/6)=**0.8** (80% parallel)

STORAGE / THROUGHPUT SUMMARY

```
Disk: random I/O = seek + rot + transfer  (very
slow)
SSD:  no seek/rot — ~10-100× faster random I/O
than disk
DRAM: ~10-100× faster than SSD
Cache: ~10× faster than DRAM

Useful BW = data_needed /
time_to_read_everything
  (row-store reading 1 of N cols → useful BW =
device_BW / N)
```

MCQ CONCEPT KEYS

- **Scheduling goal:** maximize CPU util / multiprogramming — NOT speed up one process
- **Response time low =** important for **interactive** jobs. Batch jobs → minimize turnaround
- **Processes don't share** virtual addr space. DO share: physical RAM, CPU, disk
- **Cache hit rate** primarily set by cache **SIZE** (larger → fewer misses). Not DRAM speed
- **Timer interrupt** = how OS reclaims CPU from running process without its cooperation
- **"2's complement of 12"** = representation of −12: flip(00001100)+1 = **11110100**
- **ABCD∞ LRU 3 frames:** 100% fault rate — 4 unique pages > 3 frames, always evicts
- **ABCD∞ OPT 3 frames:** ≈33% fault rate — OPT can load ABC, strategically defer D
- **fork() child:** child=0 in child process, child=PID in parent. Both resume after fork()
- **Larger pages:** ↑ internal frag, ZERO external frag (paging always uses fixed-size frames)
- **Thrashing** → ↓ CPU utilization (CPU spending all time on page faults, not useful work)
- **SJF:** selects shortest at time CPU is free. Does NOT preempt running job even if shorter arrives
- **SRTF preempts** when new arrival's burst < current job's REMAINING time (not original burst)